

Filtros más comunes en procesamiento digital de imágenes



Los filtros se usan principalmente para reducir ruido o suavizar imágenes, preservando (o no) los bordes según el tipo de filtro.

Todos estos filtros realizan un proceso de correlación (corrección) entre la imagen y un kernel 3x3 que se desliza por toda la imagen. El valor obtenido reemplaza al píxel central.

1 FILTRO DE PROMEDIO (Media o box filter)

Reemplaza cada píxel por el promedio aritmético de los valores en su vecindad.

Kernel 3x3 (todos los pesos iguales)

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

$$\sum \text{pesos} = 1$$

Mantiene el nivel promedio de brillo.

Ejemplo de efecto



Imagen original (con ruido)



Después del filtro de promedio

OpenCV (Python)

```
import cv2
# ksize debe ser impar: 3, 5, 7, ...
ksize = 3
img_promedio = cv2.blur(img, (ksize, ksize))
# También se puede usar:
# img_promedio = cv2.boxFilter(img, -1, (ksize, ksize))
```

Parámetro principal: ksize → tamaño del kernel (ventana).

2 FILTRO GAUSSIANO

Utiliza una distribución gaussiana para asignar mayor peso a los píxeles cercanos al centro.

Kernel 3x3 (ejemplo)

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

$$\sum \text{pesos} = 1$$

Suaviza la imagen con menos efecto de borde que el promedio.

Ejemplo de efecto



Imagen original (con ruido)



Después del filtro gaussiano

OpenCV (Python)

```
import cv2
# ksize debe ser impar: 3, 5, 7, ...
ksize = 3
sigma = 1.0 # Desviación estándar
img_gauss = cv2.GaussianBlur(img, (ksize, ksize), sigmaX=sigma)
# sigmaY se asume igual a sigmaX si no se especifica
```

Parámetro principal: ksize → tamaño del kernel, SigmaX (σ) → controla el grado de suavizado.

3 FILTRO DE MEDIANA (Caso especial)

Reemplaza cada píxel por la mediana de los valores en su vecindad. Es muy efectivo contra ruido sal y pimienta.

¿Cómo funciona (vecindad 3x3)?

6	11	7
13	255	9
10	6	12

Ordenamos los 9 valores:

6, 7, 8, 9, 10, 11, 12, 13, 255

Mediana = 10

Ejemplo de efecto (ruido sal y pimienta)



Imagen con ruido sal y pimienta



Después del filtro de mediana

OpenCV (Python)

```
import cv2
# ksize debe ser impar: 3, 5, 7, ...
ksize = 3
img_mediana = cv2.medianBlur(img, ksize)
```

Parámetro principal: ksize → tamaño del kernel (ventana).

4 FILTRO BILATERAL

Suaviza la imagen como el filtro gaussiano, pero preserva mejor los bordes al considerar también la diferencia de intensidad.

Idea principal

●	●	●	●	●
●	●	●	●	●
●	●	●	●	●
●	●	●	●	●
●	●	●	●	●

- Píxeles cercanos y similares (menor peso)
- Píxeles cercanos pero diferentes (menor peso)
- Píxeles lejanos o muy diferentes (peso = 0)

Ejemplo de efecto (preserva bordes)



Imagen con ruido



Después del filtro bilateral (bordes nítidos)

OpenCV (Python)

```
import cv2
# d: diámetro del vecindario
# sigmaColor: diferencia máxima de color (intensidad)
# sigmaSpace: distancia máxima en el espacio
img_bilateral = cv2.bilateralFilter(img, d=5, sigmaColor=75, sigmaSpace=75)
```

Parámetros principales: d, sigmaColor, sigmaSpace.

En resumen



Promedio: muy simple y rápido, pero tiende a borrar bordes.



Gaussiano: suaviza de forma suave y natural, con menos pérdida de detalle.



Mediana: excelente para ruido sal y pimienta, preserva mejor los bordes que los anteriores.



Bilateral: reduce ruido y conserva bordes, ideal cuando queremos suavizar sin perder estructuras importantes.

La elección del filtro depende del tipo de ruido y del objetivo: mejorar la imagen o no preservar los bordes.

